Software Requirements Specification

Assignment 1 — Full Team Deliverable

Team: CU#

UBB-SE-2026

2026

Module	Owner
Main Catalog & Filtering	**Petru**
External Reviews Dashboard	**Filip**
Standard Movie Rating & Review	**Bogdan**
Points & Badge System	**Lucas**
Weekly Battle	**Stefana**
Movie Forum	David
Voting and Discovery	Stefana

1. Main Catalog & Personalized Filtering

Owner: CU# Team

1.1 Core Catalog Display

ID	REQ-1.1	**Owner**	**Petru**
Requirement	The system shall display a primary catalog of available movies.		
Details	Each entry in the catalog must present essential movie metadata, including the title, release year, movie poster, and a summarized internal rating.		
Verifiability	A user navigating to the main view can see a populated list or grid of movie entities with all required metadata fields visible.		

1.2 Dynamic Filtering Engine

ID	REQ-1.2	**Owner**	**Petru**
Requirement	The system shall allow users to filter the displayed movie catalog based on specific metadata attributes combined with searched words.		
Details	The interface must provide selectable options to filter the catalog by Genre, associated Actors, and a minimum Review Score. A text search field must also be available.		
Verifiability	Selecting a filter option updates the catalog view to exclusively show movies matching the selected criteria. Combining multiple filters narrows results further.		

1.3 Cross-Team Integration: Personalized Recommendations

ID	REQ-1.3	**Owner**	**Petru**
Requirement	The system shall order the movie catalog based on the logged-in user's individual preferences.		
Details	The system will query the Profiling Team's data to extract the user's preferred genres and actors, applying a sorting algorithm to prioritize matching movies. The user shall have a checkbox option to enable/disable personality-based sorting.		
Verifiability	Enabling the personalization checkbox reorders the catalog based on user preference data; disabling it reverts to default ordering.		
Cross-Team	Requires read access to user preference data managed by the User Personality and Profiling Team.		

1.4 Cross-Team Integration: Entity Sharing (Movies Table)

ID	REQ-1.4	Owner	**Petru**
Requirement	The system shall act as the primary authoritative source for the Movies data entities within the shared application database.		
Details	The MovieId primary key will be used as a foreign key by the Rentals Team and Movie Events Team to link rental transactions and events to the correct film entities.		
Verifiability	Other teams can successfully create records referencing MovieId foreign keys without integrity violations.		
Cross-Team	Dependency: Rentals Team and Movie Events Team must reference this team's MovieId as a foreign key.		

2. Verified External Consensus Dashboard

Owner: Filip (Review/Rating Team)

2.1 External Data Aggregation

ID	REQ-2.1	Owner	Filip
Requirement	The system will asynchronously fetch professional review data and consensus scores for a selected movie from trusted external APIs.		
Details	The data fetch must occur without blocking the main user interface. The system must normalize data from different external sources into a standard internal format.		
Verifiability	Network logs demonstrate asynchronous outbound requests to external APIs upon loading a movie details view, resulting in populated internal data structures without UI freeze.		

2.2 Consensus & Critic Review Presentation

ID	REQ-2.2	Owner	Filip
Requirement	The system shall display the aggregated external data distinct from the internal user reviews.		
Details	The system must display numerical aggregate scores (e.g., general audience score vs. critic score). It must also display a collection of individual critic reviews with the publication source, headline, a text snippet, and a navigation link to the full external article.		
Verifiability	The UI successfully renders the aggregated scores and the list of external review snippets for a given movie, visually distinct from internal user reviews.		

2.3 Lexicon Frequency Analysis

ID	REQ-2.3	Owner	Filip
Requirement	The system shall analyze the text snippets from fetched external reviews to determine the most frequently used descriptive words.		
Details	The system must filter out standard stop-words. The top recurring words will be presented visually to the user to summarize critical sentiment.		
Verifiability	The system correctly outputs the highest-frequency non-stop-words based on the provided review text. A visual representation (e.g., word cloud or ranked list) is rendered in the UI.		

2.4 Polarization Detection System

ID	REQ-2.4	Owner	Filip
Requirement	The system shall calculate the numerical difference between the professional critic aggregate score and the general audience aggregate score.		
Details	If the calculated difference exceeds a predefined threshold, the system shall display a visual indicator notifying the user that the film has highly polarized reception. If the difference is below the threshold, this indicator must remain hidden.		
Verifiability	Manually mocking a score difference above the threshold triggers the visibility of the UI indicator; mocking a score below the threshold hides it.		

3. Standard Movie Rating & Review System

Owner: Review Team

3.1 Core Rating & Review Submission

ID	REQ-3.1	**Owner**	Review Team
Requirement	A registered user must be able to rate a movie using a 0–5 star rating system; submitting a rating doesnt require the user to also write a review.		
Details	Rating scale: 0 to 5 stars with half-star (0.5) increments. Review text: max 5000 characters. A user may rate and review a movie only once but may update or remove their submission. After a user rating, the average movie rating updates accordingly. Rating is mandatory but review is optional.		
Verifiability	A rating+review submission is stored successfully. Reviews outside the 5000 characters are rejected. The movie's average rating reflects the new submission.		

3.2 Extra Category Reviews

ID	REQ-3.2	**Owner**	Review Team
Requirement	When a user submits a standard review, the system shall allow additional ratings(mandatory) and reviews(optional) for specific movie aspects as part of an Extra Review which are all mandatory. Extra review needs an additional minim 500 to 12000 characters review blog style.		
Details	Categories: Cinematography, Acting, CGI, Plot, Sound/Music. Each category follows the same 0–5 star (0.5 increments) rating and 50–2000 character review text rules. A user may rate each category only once but may update or remove their rating.		
Verifiability	Each category needs to be rated and reviewed. Category scores are stored and associated with the correct movie and user.		

3.3 Actor Profile Awards & Badges

ID	REQ-3.4	**Owner**	Review Team
Requirement	The system shall generate badges for actor profiles highlighting top performers in specific genres based on user review data.		
Details	Profiles are automatically generated from aggregated category review data. Actor score is computed as the average acting rating from all reviews of movies the actor appears in for a given genre. Only actors meeting minimum movie count and rating thresholds (e.g., average >= 4.5 stars) receive badges. Profiles update automatically when new reviews are submitted.		
Verifiability	Actors meeting all threshold criteria display the correct genre badge on their profile page. Badge updates when new qualifying reviews are added.		

4. Gamification: Points & Badge System

Owner: Lucas

4.1 Objectivity Score Calculation

ID	REQ-4.1	Owner	Lucas
Requirement	The system shall calculate a weekly objectivity score for each user based on their review activity.		
Details	Scoring rules: +2 points for reviewing a film with avg rating > 3.5; -1 point for reviewing a film with avg rating < 2.0; +5 bonus for reviewing within one week of release (Early Bird).		
Verifiability	After each review submission, the user's score updates according to the formula. After 7 days of inactivity, the penalty is applied. Score never goes lower than zero.		

4.2 Badge / Achievement System

ID	REQ-4.2	Owner	Lucas
Requirement	The system shall automatically check and award achievement badges when a user meets predefined milestone conditions.		
Details	Badges: 'The Snob' (min 10 extra/mini-article reviews for films with avg rating < 5.0); 'The Super Serious' (all extra fields completed for 50 films); 'The Comedy Watcher' (over 70% of watch history is comedy); 'The Veteran' (100, 200, or 300 films watched). Badge checks run automatically after each relevant user action.		
Verifiability	When a user meets the criteria for a badge, the badge is unlocked, stored in the database, and a pop-up notification is displayed.		

4.3 Cross-Team: Points API for Betting

ID	REQ-4.3	Owner	Lucas
Requirement	The PointService shall expose methods to freeze and refund points, to be called by the Weekly Battle module.		
Details	FreezePoints(userId, amount): validates user has sufficient points, deducts them immediately, and marks them as frozen (unavailable for other use). RefundPoints(userId, amount): returns frozen points if a bet is cancelled.		
Verifiability	Calling FreezePoints with a valid amount correctly reduces the user's available balance. Calling with an amount exceeding the balance is rejected with an appropriate error.		
Cross-Team	Dependency: Weekly Battle module (Stefana) calls FreezePoints and DistributePayouts via the PointService interface.		

5. Weekly Battle System

Owner: Stefana (Betting) + Lucas (Points)

5.1 Battle Management (Admin/Backend)

ID	REQ-5.1	**Owner**	Stefana
Requirement	An automated or admin-triggered function shall select two movies each Monday to form a weekly battle.		
Details	A Battle table stores Movie_A_ID, Movie_B_ID, Start_Date, End_Date, and Status (Active/Finished). Logic must prevent unfair matchups. Only one battle may be active at a time.		
Verifiability	A new battle record is created every Monday. Only movies meeting eligibility criteria are selected. The battle status transitions from Active to Finished at end of the week.		

5.2 Betting Engine

ID	REQ-5.2	**Owner**	Stefana
Requirement	Users must be able to stake a portion of their points on one of the two battling movies.		
Details	Validation ensures the user has sufficient points. Points are deducted immediately via FreezePoints(). The user's bet is stored: User_ID, Battle_ID, Chosen_Movie_ID, Amount_Staked. A user may only place one bet per battle.		
Verifiability	Placing a valid bet deducts points and stores the record. Attempting to bet more points than available is rejected. A second bet attempt on the same battle is rejected.		
Cross-Team	Dependency: Lucas (PointService) — FreezePoints(userId, amount) must be available.		

5.3 Winner Determination & Payout

ID	REQ-5.3	**Owner**	Stefana
Requirement	At the end of the week, the system shall determine the winner based on rating increase (ΔR) during the battle period and distribute payouts.		
Details	Winner = movie with highest ΔR during battle period. Payout formula: Points Won = Stake × Multiplier (fixed or dynamic based on underdog bets). A Sunday midnight script identifies winners and triggers PointService to add points to winning users' profiles.		
Verifiability	After the battle ends, winners' point balances correctly reflect their payout. Losing bettors' frozen points are not returned.		
Cross-Team	Dependency: Lucas (PointService) — DistributePayouts must trigger point addition via PointService.		

5.4 Battle UI (Arena View)

ID	REQ-5.4	Owner	Stefana
Requirement	A dedicated page shall show the two competing movies side-by-side with a Bet Now interface.		
Details	The view displays both movies and a bet input.		
Verifiability	The Arena View renders both movies. Placing a bet from the UI correctly triggers the betting engine.		

6. Movie Forum Module

Owner: David

6.1 Thread Creation

ID	REQ-6.1	**Owner**	Forum Team
Requirement	Users must be able to create discussion threads linked to a specific movie.		
Details	The thread creation interface must allow the user to input: Title (short text field, max 500 characters), Content (long text field describing the discussion topic, max 10000) Thread contains: Title, Content, MovieId (FK) - automatically assigned based on the movie page where the thread is created, AuthorId (FK) - automatically assigned based on the logged-in user , CreationDate - automatically generated by the system, Score - initialized to 0 . Thread appears in the movie's discussion list upon creation.		
Verifiability	After submitting a valid thread, it appears in the movie's thread list. The thread is retrievable from the database with correct MovieId and AuthorId.		
Cross-Team	Dependency: MovieId from Marketplace/CU# Team; UserId from Personality Matching Team.		

6.2 Thread & Comment Viewing

ID	REQ-6.2	**Owner**	Forum Team
Requirement	Users must be able to view all threads associated with a movie and sort them using different criteria. Users must also be able to open a thread to view its full discussion.		
Details	Sorting options: Newest, Most Upvoted, Hottest (gravity algorithm). Users may click a thread to view its comments.		
Verifiability	All threads for a movie are displayed. Each sort option correctly reorders the thread list. Clicking a thread navigates to its detail view.		

6.3 Comments & Nested Replies

ID	REQ-6.3	**Owner**	Forum Team
Requirement	Users must be able to comment on threads and reply to existing comments (nested replies).		
Details	Comment contains: Content – max 10000, AuthorId - automatically assigned based on the logged-in user, ThreadId (FK) – automatically assigned based on the thread the user is commenting on, Timestamp – automatically generated by the system, Score, and optional ParentCommentId (FK, nullable) – based if the user is commenting on a comment other than a thread. Replies are displayed nested under their parent comment.		
Verifiability	A comment without a parent appears at the root level. A reply with a ParentCommentId appears nested under the correct parent comment.		

7. Voting & Discovery

Owner: Dalalau

6.4 Upvoting System

ID	REQ-6.4	**Owner**	Forum Team
Requirement	Users must be able to upvote threads and comments. Each user may upvote a given item only once.		
Details	Upvoting a thread or comment increments its Score immediately. The Vote table enforces uniqueness: (UserId + ThreadId) and (UserId + CommentId) must be unique.		
Verifiability	A user can upvote an item once; a second attempt is rejected. Score updates are reflected immediately in the UI.		

6.5 Hottest Threads Algorithm

ID	REQ-6.5	**Owner**	Forum Team
Requirement	The system shall calculate a HotScore for each thread to surface trending discussions.		
Details	Formula: HotScore = Score / (hoursSinceCreation + 2)^1.5. Threads are sorted by this value when the 'Hottest' option is selected.		
Verifiability	New threads with high scores appear near the top. Older threads with the same score rank lower. The sort correctly applies the gravity formula.		

7. Task Breakdown

All tasks are scoped to a maximum of 30 minutes of work. Total tasks: 70+

7.1 Main Catalog & Filtering Tasks

Task Name	Description / Acceptance Criteria	Est. Time
Define Movie domain model	Create C# Movie class with all required properties (Id, Title, Year, PosterUrl, AverageRating, GenreId, etc.).	20m
Configure Movies SQL table	Write SQL migration/script for Movies table with correct columns, PK, and indexes.	25m
Implement MovieRepository.GetAll()	Write data access method to retrieve all movies from the DB.	20m
Implement filter by Genre	Add Genre filter parameter to GetMovies() query. Returns only movies matching the genre.	20m
Implement filter by Actor	Add Actor filter. Query movies via join with Actors table.	25m
Implement filter by minimum Score	Add MinScore filter parameter. Returns movies with AverageRating >= threshold.	15m
Implement text search filter	Add keyword search over movie titles. Case-insensitive LIKE query.	20m
Create CatalogViewModel	Build ViewModel with ObservableCollection<Movie> and filter properties.	30m
Implement sort by user preference	Add sorting logic that reorders catalog based on user preferred genres/actors from Profiling Team data.	30m
Add personalization toggle (checkbox)	Implement ICommand for enable/disable personalized sort. Updates catalog ordering on toggle.	20m
Design XAML Catalog View	Create main catalog grid/list XAML with poster, title, year, rating. Bind to CatalogViewModel.	30m
Design XAML Filter Panel	Add filter sidebar/panel in XAML with genre dropdown, actor input, score slider, and text search box.	30m

7.2 External Reviews Dashboard Tasks (Filip)

Task Name	Description / Acceptance Criteria	Est. Time
Define ExternalReview model	Create C# class: Source, Headline, Snippet, Url, CriticScore, AudienceScore.	15m
Implement async API fetch (Rotten Tomatoes / IMDB)	Write async method to call external API. Must not block UI thread.	30m
Implement data normalization	Write mapper to convert API response into internal ExternalReview format.	25m
Implement score aggregation	Compute average critic score and average audience score from fetched data.	20m
Implement lexicon frequency analyzer	Write algorithm: tokenize review snippets, filter stop-words, count word frequency, return top N words.	30m
Implement polarization detector	Calculate |criticScore - audienceScore|. If > threshold, set IsPolarized = true.	20m
Create ExternalReviewViewModel	ViewModel exposing ExternalReviews list, AggregateCriticScore, AggregateAudienceScore, TopWords, IsPolarized.	30m
Design XAML: Score display section	Render critic vs audience score bars/gauges. Bind to ViewModel.	30m
Design XAML: Review snippets list	Render list of critic reviews: source, headline, snippet, link button.	25m
Design XAML: Lexicon word cloud / list	Display top words visually (frequency bars or tag cloud).	25m
Design XAML: Polarization indicator	Conditionally visible banner/icon when IsPolarized = true.	15m

7.3 Standard Rating & Review Tasks

Task Name	Description / Acceptance Criteria	Est. Time
Define Review domain model	C# class: ReviewId, UserId, MovieId, StarRating (decimal 0–5), Text, CreatedAt, UpdatedAt.	15m
Define ExtraReview domain model	C# class: ExtraReviewId, UserId, MovieId, Category (enum), StarRating, Text.	15m
Configure Reviews SQL tables	SQL scripts for Reviews, ExtraReviews tables with correct PKs and FKs.	30m
Implement ReviewService.SubmitReview()	Validate rating range, text length (50–2000). Reject if missing either field. Save to DB.	30m
Implement ReviewService.UpdateReview()	Allow user to edit existing review. Re-validate constraints.	20m
Implement ReviewService.DeleteReview()	Remove review from DB. Trigger average recalculation.	15m
Implement average rating recalculation	After any review add/update/delete, recompute and store movie average rating.	25m

Implement ExtraReviewService.Submit()	Validate category, rating, and text (50–2000). Store category-specific review.	**25m**
Implement ActorBadgeService	Compute avg acting rating per actor per genre from category reviews. Award badge if >= threshold.	**30m**
Create ReviewViewModel	ViewModel for rating input (0–5, 0.5 steps), review text, submit/update/delete commands.	**30m**
Design XAML: Rating input (stars)	Star rating control with 0.5 increment selection. Bind to ViewModel.	**30m**
Design XAML: Review text input & validation	Text area with character counter, error messages for invalid lengths.	**25m**
Design XAML: Extra category review panel	One rating+text input per category. Collapsible section.	**30m**

7.4 Points & Badge System Tasks (Lucas)

Task Name	Description / Acceptance Criteria	Est. Time
Define UserStats domain model	C# class: UserLocalId, TotalPoints, WeeklyScore, LastActive.	**15m**
Define Badge domain model	C# class: BadgeId, Name, CriteriaValue. Implement ToString().	**15m**
Configure UserStats SQL table	SQL script for LocalUserStats table with PK and constraints (Points >= 0).	**20m**
Configure Badges SQL tables	SQL scripts for BadgesMasterList and UserToBadges (many-to-many with DateUnlocked).	**25m**
Script to seed badge master list	SQL INSERT statements for all 4 badges with their criteria values.	**20m**
Create IPointService interface	Define interface with AddPoints, FreezePoints, RefundPoints.	**15m**
Implement AddPoints(double rating)	Apply scoring rules: +2 if rating > 3.5, -1 if rating < 2.0, +5 Early Bird bonus.	**30m**
Implement FreezePoints(userId, amount)	Validate balance, deduct points, mark as frozen. Return error if insufficient.	**25m**
Implement RefundPoints(userId, amount)	Return frozen points to available balance.	**20m**
Implement inactivity penalty check	Check if LastActive > 7 days ago; if so, apply -2 penalty.	**25m**
Implement badge check: The Snob	Count extra mini-article reviews for films with avg < 5.0. Award if >= 10.	**30m**
Implement badge check: The Super Serious	Count movies where all extra fields are complete. Award if >= 50.	**30m**
Implement badge check: Comedy Watcher	Compute % of watch history that is comedy. Award if > 70%.	**30m**
Implement badge check: The Veteran	Count total films watched. Award badge at 100, 200, 300 milestones.	**25m**

Implement GetUnlockedBadges(userId)	Return list of badges the user has earned from UserToBadges.	**15m**
Create ProfileViewModel	ObservableCollection<Badge>, DisplayPoints property with OnPropertyChanged.	**30m**
Design XAML: Points display	Show TotalPoints, WeeklyScore in profile view.	**25m**
Design XAML: Badge grid	Grid layout showing all earned badges with icon and name.	**30m**
Implement badge unlock pop-up	Pop-up notification: 'Congratulations! You unlocked [Badge Name]'.	**25m**

7.5 Weekly Battle Tasks (Stefana)

Task Name	Description / Acceptance Criteria	Est. Time
Define Battle domain model	C# class: BattleId, MovieAId, MovieBId, StartDate, EndDate, Status (enum Active/Finished).	**10m**
Define UserBet domain model	C# class: BetId, UserId, BattleId, ChosenMovieId, AmountStaked.	**10m**
Configure Battles SQL table	SQL migration for Battles table with all fields and Status constraint.	**20m**
Configure UserBets SQL table	SQL migration for UserBets table with FK constraints.	**20m**
Implement BattleService.SelectWeeklyMovies()	Select 2 eligible movies for the week. Create a new Battle record with Active status.	**30m**
Implement FreezePoints call on bet	When user places bet, call PointService.FreezePoints(userId, amount) before storing the bet.	**25m**
Implement PlaceBet(userId, movieId, amount)	Validate user has sufficient points, call FreezePoints, store UserBet record.	**30m**
Implement winner calculation (ΔR)	At battle end, compute rating delta for both movies during battle period. Higher ΔR wins.	**30m**
Implement DistributePayouts()	For all winning bets: calculate Points Won = Stake × Multiplier. Call PointService.AddPoints.	**30m**
Create BattleViewModel	Properties: MovieA, MovieB, LiveOdds, CurrentLeader, UserBetAmount, BetCommand.	**30m**
Implement Movie A and B properties	Bind MovieA and MovieB with title, poster, current rating in BattleViewModel.	**20m**
Design XAML: Versus screen	Side-by-side movie display with Bet Now input and submit button.	**30m**
Design XAML: Live progress bar	ProgressBar showing which movie is currently leading in ratings.	**30m**
Implement bet ICommand	ICommand bound to Bet Now button. Calls PlaceBet with current input.	**20m**
Implement battle result pop-up	Pop-up notification at battle end: show winner and user's result (win/loss).	**25m**

7.6 Movie Forum Tasks

Task Name	Description / Acceptance Criteria	Est. Time
Create Thread domain model	C# class: ThreadId, MovieId (FK), AuthorId (FK), Title, Content, CreatedAt, Score.	30m
Create Comment domain model	C# class: CommentId, ThreadId (FK), AuthorId (FK), Content, ParentCommentId (nullable FK), CreatedAt, Score.	20m
Create Vote domain model	C# class: VoteId, UserId, ThreadId (nullable), CommentId (nullable), Value, CreatedAt.	15m
Configure Entity Framework DB Context	Add DbSet for Threads, Comments, Votes. Configure FKs and unique constraints (UserId+ThreadId, UserId+CommentId).	30m
Implement ThreadService.CreateThread()	Validate and save new thread. Attach MovieId, AuthorId. Set CreatedAt = DateTime.UtcNow.	30m
Implement ThreadService.GetThreadsByMovie()	Retrieve all threads filtered by MovieId.	20m
Implement CommentService.AddComment()	Save new comment linked to ThreadId.	20m
Implement CommentService.GetThreadComments()	Fetch all comments for a specific ThreadId.	20m
Implement CommentService.ReplyToComment()	Create comment with same ThreadId as parent and set ParentCommentId.	30m
Implement CommentService.BuildCommentTree()	Transform flat List<Comment> into hierarchical parent-child tree structure.	30m
Implement VoteService: CheckExistingVote() + AddVote()	Prevent duplicate voting via DB constraint check. Update target Score immediately.	30m
Implement HotScore gravity formula	Write utility: HotScore = Score / (hoursSinceCreation + 2)^1.5.	20m
Implement ThreadService.GetHotThreads()	Apply gravity formula to thread list and return sorted by HotScore.	30m
Create MovieForumViewModel & View	ObservableCollection<Thread>, sort commands (Hot/New/Top), Create button. Draft XAML for MovieForumView.	30m
Create ThreadDetailViewModel & View	ViewModel with CurrentThread and nested Comments. XAML with upvote/reply buttons.	30m
Create CreateThreadViewModel & View	Title/Content input ViewModel with SubmitCommand. XAML with text boxes and submit button.	30m

8. Cross-Team Dependencies Summary

Consumer Module	Provides	Data / Method	Description
Catalog (CU#)	Personality/Profiling Team	**User preference data**	Read access to preferred genres/actors for catalog sorting.
Rentals Team	Catalog (CU#)	**MovieId (FK)**	Rental records reference MovieId from Movies table.
Events Team	Catalog (CU#)	**MovieId (FK)**	Event records reference MovieId from Movies table.
Weekly Battle (Stefana)	Points (Lucas)	**FreezePoints(userId, amount)**	Deduct & freeze points when a bet is placed.
Weekly Battle (Stefana)	Points (Lucas)	**DistributePayouts()**	Add points to winners after battle ends.
Points (Lucas)	Movies (CU#)	**MovieId, ReleaseDate**	Required for Early Bird bonus calculation.
Points (Lucas)	Users (Profiling)	**UserId**	Associate points with correct user record.
Forum Team	Movies (CU#)	**MovieId (FK)**	Forum threads reference MovieId for movie-specific discussions.
Forum Team	Users (Profiling)	**UserId (FK)**	Thread/comment AuthorId references the user system.

9. Database Schema Overview

The following tables are defined across all modules. Cross-team references use Foreign Keys as noted.

Core Tables

- Movies (CU# Team) — MovieId PK, Title, Year, PosterUrl, AverageRating, GenreId
- Users — UserId PK (managed by Personality Matching Team)
- Reviews — ReviewId PK, UserId FK, MovieId FK, StarRating, Text, CreatedAt
- ExtraReviews — ExtraReviewId PK, UserId FK, MovieId FK, Category, StarRating, Text
- ExternalReviews — ExternalReviewId PK, MovieId FK, Source, Headline, Snippet, Url, CriticScore, AudienceScore
- UserStats — UserLocalId PK, TotalPoints, WeeklyScore, LastActive
- Badges — BadgeId PK, Name, CriteriaValue
- UserBadges — UserLocalId FK, BadgeId FK, DateUnlocked (composite PK)
- Battles — BattleId PK, MovieAId FK, MovieBId FK, StartDate, EndDate, Status
- UserBets — BetId PK, UserId FK, BattleId FK, ChosenMovieId FK, AmountStaked
- Threads — ThreadId PK, MovieId FK, AuthorId FK, Title, Content, CreatedAt, Score
- Comments — CommentId PK, ThreadId FK, AuthorId FK, Content, ParentCommentId FK (nullable), CreatedAt, Score
- Votes — VoteId PK, UserId FK, ThreadId FK (nullable), CommentId FK (nullable), Value, CreatedAt

Key Relationships

- Movie → Thread: 1 to Many
- Movie → Review / ExtraReview : 1 to Many
- User → Thread: 1 to Many
- User → Comment: 1 to Many
- Thread → Comment: 1 to Many
- Comment → Comment: 1 to Many (self-referencing nested replies)
- User → UserBets: 1 to Many
- Battle → UserBets: 1 to Many
- User → UserBadges: Many to Many (via UserBadges join table)

10. Architecture & Design Notes

All modules follow the MVVM (Model-View-ViewModel) pattern.

Pattern

- Model (Domain Layer): C# domain classes, EF DbContext, SQL tables.
- ViewModel (Business Logic): Services (ThreadService, PointService, ReviewService, BattleService, etc.), ViewModels with ObservableCollection and ICommand.
- View (UI Layer): XAML views with data binding. No business logic in code-behind.

Multiple Views Required

- CatalogView — main movie catalog with filters
- MovieDetailView — movie page hub (ratings, reviews, external consensus, forum)
- ExternalReviewView — critic consensus and lexicon analysis
- ReviewSubmitView — rating and review entry
- ProfileView — user points and badges
- BattleArenaView — weekly battle versus screen
- BattleHistoryView — past bets history
- MovieForumView — thread list for a movie
- ThreadDetailView — thread content and comments
- CreateThreadView — new thread form